

Web Development Lecture Notes

DOM Events, Form Validation, Git, Debugging & Testing

Table of Contents

1. DOM Events
 - Form-based Events
 - Keyboard and Mouse Events
2. JavaScript Form Validation
3. Git Basics and Version Control Workflow
4. Debugging Techniques Using Browser Tools
5. Error Handling in JavaScript
6. Introduction to Unit Testing Concepts

Part 1: DOM Events

What are DOM Events?

- **DOM Events** are signals that something has occurred (user interaction, browser action)
- Events allow JavaScript to react to user actions
- The **EventTarget** interface provides `addEventListener()`

```
// Basic syntax  
element.addEventListener(event, function, useCapture);
```

Event Listener Example

```
<!DOCTYPE html>
<html>
<head>
  <title>Event Demo</title>
</head>
<body>
  <button id="myBtn">Click Me!</button>
  <p id="demo"></p>

  <script>
    const btn = document.getElementById('myBtn');
    const demo = document.getElementById('demo');

    btn.addEventListener('click', function() {
      demo.textContent = 'Button was clicked!';
      demo.style.color = 'green';
    });
  </script>
</body>
</html>
```

Removing Event Listeners

```
function handleClick() {  
  console.log('Clicked!');  
}  
  
// Add listener  
btn.addEventListener('click', handleClick);  
  
// Remove listener (must use named function)  
btn.removeEventListener('click', handleClick);
```

Event Object

```
btn.addEventListener('click', function(event) {  
  console.log(event.type);      // "click"  
  console.log(event.target);    // the element clicked  
  console.log(event.clientX);   // X coordinate  
  console.log(event.clientY);   // Y coordinate  
  event.preventDefault();       // prevent default behavior  
  event.stopPropagation();      // stop bubbling  
});
```

1.1 Form-Based Events

Common Form Events

Event	Description
submit	Form is submitted
reset	Form is reset
change	Value of input changes (after blur)
input	Value changes immediately
focus	Element receives focus
blur	Element loses focus
select	Text is selected

Form Submit Event

```
<!DOCTYPE html>
<html>
<body>
  <form id="myForm">
    <input type="text" name="username" placeholder="Username">
    <input type="password" name="password" placeholder="Password">
    <button type="submit">Login</button>
  </form>

  <script>
    const form = document.getElementById('myForm');

    form.addEventListener('submit', function(event) {
      event.preventDefault(); // Stop actual submission

      const formData = new FormData(form);
      const data = Object.fromEntries(formData);

      console.log('Form Data:', data);
      // { username: "john", password: "secret" }
    });
  </script>
</body>
</html>
```

Change vs Input Events

```
<input type="text" id="nameInput" placeholder="Type here...">
<p id="changeResult"></p>
<p id="inputResult"></p>

<script>
  const input = document.getElementById('nameInput');
  const changeResult = document.getElementById('changeResult');
  const inputResult = document.getElementById('inputResult');

  // Fires when value changes AND element loses focus
  input.addEventListener('change', function() {
    changeResult.textContent = 'Change: ' + this.value;
  });

  // Fires on EVERY keystroke
  input.addEventListener('input', function() {
    inputResult.textContent = 'Input: ' + this.value;
  });
</script>
```

Focus and Blur Events

```
<input type="email" id="email" placeholder="Email">
<span id="emailError" style="color: red;"></span>

<script>
  const email = document.getElementById('email');
  const error = document.getElementById('emailError');

  email.addEventListener('focus', function() {
    this.style.borderColor = 'blue';
    error.textContent = '';
  });

  email.addEventListener('blur', function() {
    if (!this.value.includes('@')) {
      this.style.borderColor = 'red';
      error.textContent = 'Please enter a valid email';
    } else {
      this.style.borderColor = 'green';
    }
  });
</script>
```

Select Event

```
<textarea id="story" rows="4" cols="50">
Once upon a time...
</textarea>
<p id="selection"></p>

<script>
  const story = document.getElementById('story');
  const selection = document.getElementById('selection');

  story.addEventListener('select', function() {
    const selectedText = window.getSelection().toString();
    selection.textContent = 'Selected: ' + selectedText + ' ';
  });
</script>
```

1.2 Keyboard and Mouse Events

Keyboard Events

Event	Description
keydown	Key is pressed down
keyup	Key is released
keypress	Key is pressed (deprecated, use keydown)

Keyboard Event Example

```
<input type="text" id="search" placeholder="Search...">
<p id="keyInfo"></p>

<script>
  const search = document.getElementById('search');
  const keyInfo = document.getElementById('keyInfo');

  search.addEventListener('keydown', function(event) {
    keyInfo.innerHTML = `
      Key: ${event.key}<br>
      Code: ${event.code}<br>
      Ctrl: ${event.ctrlKey}<br>
      Shift: ${event.shiftKey}<br>
      Alt: ${event.altKey}
    `;

    // Detect Enter key
    if (event.key === 'Enter') {
      console.log('Search submitted:', this.value);
    }

    // Detect Ctrl+S
    if (event.ctrlKey && event.key === 's') {
      event.preventDefault();
      console.log('Save shortcut detected!');
    }
  });
</script>
```

Mouse Events

Event	Description
<code>click</code>	Single click
<code>dblclick</code>	Double click
<code>mousedown</code>	Mouse button pressed
<code>mouseup</code>	Mouse button released
<code>mousemove</code>	Mouse moves over element
<code>mouseenter</code>	Mouse enters element
<code>mouseleave</code>	Mouse leaves element
<code>mouseover</code>	Mouse over element (bubbles)
<code>mouseout</code>	Mouse out of element (bubbles)
<code>contextmenu</code>	Right-click menu

Mouse Event Example

```
<div id="box" style="width: 200px; height: 200px; background: lightblue;">
  Hover and click me!
</div>
<p id="mouseInfo"></p>

<script>
  const box = document.getElementById('box');
  const info = document.getElementById('mouseInfo');

  box.addEventListener('mouseenter', function() {
    this.style.backgroundColor = 'lightgreen';
  });

  box.addEventListener('mouseleave', function() {
    this.style.backgroundColor = 'lightblue';
  });

  box.addEventListener('mousemove', function(event) {
    info.innerHTML = `
      X: ${event.offsetX}, Y: ${event.offsetY}<br>
      Buttons: ${event.buttons}
    `;
  });

  box.addEventListener('contextmenu', function(event) {
    event.preventDefault();
    alert('Right-click disabled!');
  });
</script>
```

Drag and Drop Events

```
<div id="draggable" draggable="true"
  style="width: 100px; height: 100px; background: coral;">
  Drag me
</div>
<div id="dropzone"
  style="width: 200px; height: 200px; border: 2px dashed gray; margin-top: 20px;">
  Drop here
</div>
```

```
<script>
const drag = document.getElementById('draggable');
const drop = document.getElementById('dropzone');

drag.addEventListener('dragstart', function(event) {
  event.dataTransfer.setData('text/plain', 'dragged');
  this.style.opacity = '0.5';
});

drag.addEventListener('dragend', function() {
  this.style.opacity = '1';
});

drop.addEventListener('dragover', function(event) {
  event.preventDefault(); // Required to allow dropping
  this.style.backgroundColor = 'lightyellow';
});

drop.addEventListener('dragleave', function() {
  this.style.backgroundColor = '';
});

drop.addEventListener('drop', function(event) {
  event.preventDefault();
  this.style.backgroundColor = 'lightgreen';
  this.textContent = 'Dropped!';
});
</script>
```

The user starts dragging the element.

The drag operation finishes (whether successful or not).

A dragged element is moved over the drop target.

A dragged element leaves the drop target's area.

The dragged element is released on the target.

Event Delegation

Event delegation is a JavaScript pattern used to handle events efficiently by attaching a single event listener to a common ancestor (parent) rather than multiple individual child elements.

1. Trigger: An event occurs on a child element (e.g., a button click).
2. Propagation: The event "bubbles up" through its ancestors in the DOM tree until it reaches the root element.
3. Intercept: A listener attached to a parent element intercepts this bubbling event.
4. Targeting: The listener identifies which specific child triggered the event using the `event.target` property.

Event Delegation Example

```
<html> <body>
<h1>Event Delegation</h1>
<ul id="myList">
  <li>Item 1</li>    <li>Item 2</li>    <li>Item 3</li>
</ul>
<script>
  // Get the parent element
  const myList = document.getElementById('myList');
  // Add event listener to the parent element
  myList.addEventListener('click', function(event) {
    // Check if the clicked element is a list item
    // event.target is the actual element that was clicked
    if (event.target && event.target.nodeName === 'LI') {
      console.log('Clicked on:', event.target.textContent);
    }
  });
</script>
</body>    </html>
```

Event Delegation Example2

```
<ul id="todoList">
  <li>Task 1 <button class="delete">X</button></li>
  <li>Task 2 <button class="delete">X</button></li>
  <li>Task 3 <button class="delete">X</button></li>
</ul>

<script>
  const list = document.getElementById('todoList');

  // Instead of adding listener to each button
  list.addEventListener('click', function(event) {
    if (event.target.classList.contains('delete')) {
      const li = event.target.closest('li');
      li.remove();
      console.log('Task deleted');
    }
  });

  // New items automatically work!
  const newItem = document.createElement('li');
  newItem.innerHTML = 'Task 4 <button class="delete">X</button>';
  list.appendChild(newItem);
</script>
```

Part 2: JavaScript Form Validation

Client-Side Validation Approaches

1. **HTML5 Built-in Validation** - Simple, no JS needed
2. **JavaScript Validation** - Custom logic, more control
3. **Server-Side Validation** - Always required for security

HTML5 Validation Attributes

```
<form>
  <!-- Required field -->
  <input type="text" required placeholder="Required">

  <!-- Pattern matching -->
  <input type="text" pattern="[A-Za-z]{3}"
    title="Three letters only">

  <!-- Min/Max length -->
  <input type="text" minlength="3" maxlength="10">

  <!-- Number range -->
  <input type="number" min="1" max="100">

  <!-- Email validation -->
  <input type="email" placeholder="Email">

  <!-- URL validation -->
  <input type="url" placeholder="Website">

  <button type="submit">Submit</button>
</form>
```

JavaScript Form Validation - Basic

```
<form id="registerForm">
  <input type="text" id="username" placeholder="Username">
  <span id="userError" class="error"></span>

  <input type="email" id="email" placeholder="Email">
  <span id="emailError" class="error"></span>

  <input type="password" id="password" placeholder="Password">
  <span id="passError" class="error"></span>

  <button type="submit">Register</button>
</form>

<script>
  const form = document.getElementById('registerForm');

  function showError(fieldId, message) {
    document.getElementById(fieldId + 'Error').textContent = message;
  }

  function clearErrors() {
    document.querySelectorAll('.error').forEach(e => e.textContent = '');
  }

  form.addEventListener('submit', function(event) {
    event.preventDefault();
    clearErrors();
    let isValid = true;

    const username = document.getElementById('username').value.trim();
    if (username.length < 3) {
      showError('user', 'Username must be at least 3 characters');
      isValid = false;
    }

    const email = document.getElementById('email').value.trim();
    const emailRegex = /^[^s@]+@[^s@]+\.[^s@]+$/;
    if (!emailRegex.test(email)) {
      showError('email', 'Please enter a valid email');
      isValid = false;
    }

    const password = document.getElementById('password').value;
    if (password.length < 8) {
      showError('pass', 'Password must be at least 8 characters');
      isValid = false;
    }

    if (isValid) {
      console.log('Form submitted:', { username, email, password });
      form.reset();
    }
  });
</script>
```

Real-Time Validation

```
<input type="password" id="pwd" placeholder="Password">
<div id="strength">
  <p id="length">X At least 8 characters</p>
  <p id="upper">X One uppercase letter</p>
  <p id="lower">X One lowercase letter</p>
  <p id="number">X One number</p>
</div>

<script>
  const pwd = document.getElementById('pwd');

  pwd.addEventListener('input', function() {
    const value = this.value;

    updateRequirement('length', value.length >= 8);
    updateRequirement('upper', /[A-Z]/.test(value));
    updateRequirement('lower', /[a-z]/.test(value));
    updateRequirement('number', /\d/.test(value));
  });

  function updateRequirement(id, valid) {
    const el = document.getElementById(id);
    el.textContent = el.textContent.replace(/^[X√]/, valid ? '√' : 'X');
    el.style.color = valid ? 'green' : 'red';
  }
</script>
```

Custom Validation with Constraint API

```
<form id="myForm">
  <input type="text" id="phone"
    placeholder="Phone (XXX-XXX-XXXX)"
    required>
  <span id="phoneError"></span>
  <button type="submit">Submit</button>
</form>

<script>
  const phone = document.getElementById('phone');

  phone.addEventListener('input', function() {
    const regex = /^\\d{3}-\\d{3}-\\d{4}$/;

    if (regex.test(this.value)) {
      this.setCustomValidity(''); // Valid
    } else {
      this.setCustomValidity('Format: 123-456-7890');
    }
  });

  // Check validity programmatically
  document.getElementById('myForm').addEventListener('submit', function(e) {
    if (!this.checkValidity()) {
      e.preventDefault();
      // reportValidity() shows browser's native tooltips
      this.reportValidity();
    }
  });
</script>
```

Part 3: Git Basics and Version Control Workflow

What is Git?

- **Git** is a distributed version control system
- Tracks changes to files over time
- Enables collaboration among developers
- Created by Linus Torvalds in 2005

Git Configuration

```
# Set your identity (do this first!)
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"

# Check configuration
git config --list

# Set default editor
git config --global core.editor "code --wait"
```

Creating a Repository

```
# Initialize a new repository
mkdir my-project
cd my-project
git init

# Or clone an existing repository
git clone https://github.com/user/repo.git
git clone https://github.com/user/repo.git my-folder
```

Basic Git Workflow

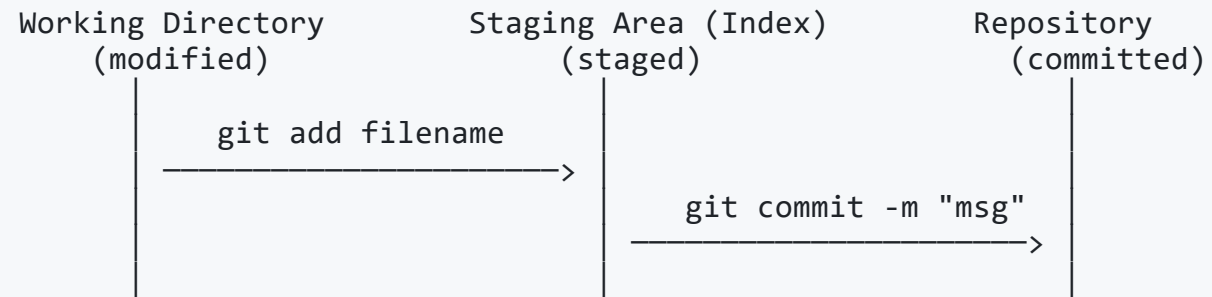
```
# 1. Check status
git status

# 2. Stage files (add to staging area)
git add filename.js      # Stage specific file
git add .                 # Stage all changes

# 3. Commit changes
git commit -m "Add user authentication feature"

# 4. View history
git log
git log --oneline        # Compact view
git log --graph --oneline # With branch graph
```

Git Staging Visualization



Understanding Git States

```
# Untracked - new file, Git doesn't know about it
git status
# → "Untracked files:"
```

```
# Modified - tracked file with changes
git status
# → "Changes not staged for commit:"
```

```
# Staged - added to index, ready to commit
git status
# → "Changes to be committed:"
```

```
# Committed - saved in repository
git status
# → "nothing to commit, working tree clean"
```

Branching

```
# List branches
git branch

# Create new branch
git branch feature-login

# Switch to branch
git checkout feature-login
# OR (newer syntax)
git switch feature-login

# Create and switch in one command
git checkout -b feature-login
git switch -c feature-login

# Merge branch into current
git checkout main
git merge feature-login
```

Branching Workflow Example

```
# Start new feature
git checkout -b feature-user-profile

# Make changes and commit
git add .
git commit -m "Add user profile page"

# Switch back to main
git checkout main

# Merge the feature
git merge feature-user-profile

# Delete the feature branch
git branch -d feature-user-profile
```

Remote Repositories

```
# Add remote repository
git remote add origin https://github.com/user/repo.git

# View remotes
git remote -v

# Push to remote
git push origin main

# Push new branch
git push -u origin feature-branch

# Fetch changes (without merging)
git fetch origin

# Pull changes (fetch + merge)
git pull origin main
```

Complete Daily Workflow

```
# Morning - get latest changes
git pull origin main

# Work on feature
git checkout -b fix-login-bug

# ... make changes ...

# Stage and commit
git add .
git commit -m "Fix login redirect issue"

# Push to remote
git push -u origin fix-login-bug

# Create Pull Request on GitHub/GitLab
# After review, merge via UI or:
git checkout main
git merge fix-login-bug
git push origin main
```

Undoing Changes

```
# Unstage a file (keep changes)
git reset HEAD filename.js

# Discard working directory changes
git checkout -- filename.js
git restore filename.js      # newer syntax

# Amend last commit
git commit --amend -m "New message"

# Revert a commit (creates new commit)
git revert abc1234

# Reset to previous commit (DANGEROUS)
git reset --hard HEAD~1
```

.gitignore File

```
# Node.js
node_modules/
npm-debug.log*
package-lock.json

# Build outputs
dist/
build/
*.min.js

# Environment variables
.env
.env.local

# IDE
.vscode/
.idea/
*.swp

# OS files
.DS_Store
Thumbs.db

# Logs
*.log
logs/
```

Part 4: Debugging Techniques Using Browser Tools

Browser DevTools Overview

- **Elements:** Inspect and edit HTML/CSS
- **Console:** View logs and execute JS
- **Sources:** Debug JavaScript
- **Network:** Monitor HTTP requests
- **Application:** Storage, cookies, service workers
- **Performance:** Profile page performance

Open with: F12 or Ctrl+Shift+I (Cmd+Option+I on Mac)

Console Methods

```
// Basic logging
console.log('Hello World');
console.warn('This is a warning');
console.error('This is an error');
console.info('Information message');

// Styled console
console.log('%c Styled Text', 'color: blue; font-size: 20px;');

// Grouping
console.group('User Data');
console.log('Name: John');
console.log('Age: 25');
console.groupEnd();

// Tables
const users = [
  { name: 'Alice', age: 25 },
  { name: 'Bob', age: 30 }
];
console.table(users);

// Timing
console.time('loop');
for (let i = 0; i < 1000; i++) {}
console.timeEnd('loop'); // loop: 0.5ms

// Assertions
console.assert(1 === 2, 'This will print');
console.assert(1 === 1, 'This will not print');
```

Setting Breakpoints in DevTools

```
// Method 1: debugger statement
function calculateTotal(items) {
  let total = 0;

  for (let item of items) {
    debugger; // Execution pauses here
    total += item.price * item.quantity;
  }

  return total;
}

// Method 2: DevTools Sources panel
// 1. Open Sources tab
// 2. Find your JS file
// 3. Click line number to set breakpoint
// 4. Refresh page to trigger
```

Debugging with Breakpoints

```
function processOrder(order) {  
  console.log('Processing order:', order.id);  
  
  // Set breakpoint on next line in DevTools  
  const items = order.items;  
  const customer = order.customer;  
  
  // Step Over (F10): Execute current line, move to next  
  // Step Into (F11): Enter function calls  
  // Step Out (Shift+F11): Exit current function  
  // Resume (F8): Continue until next breakpoint  
  
  const total = calculateTotal(items);  
  const tax = total * 0.08;  
  
  return {  
    total: total,  
    tax: tax,  
    grandTotal: total + tax  
  };  
}
```

Watching Variables

```
function factorial(n) {  
  let result = 1;  
  
  // In DevTools Sources panel:  
  // 1. Add 'n' and 'result' to Watch expressions  
  // 2. Set breakpoint on the for loop  
  // 3. Step through to see values change  
  
  for (let i = 1; i <= n; i++) {  
    result *= i; // Watch result change here  
  }  
  
  return result;  
}  
  
factorial(5); // Call to trigger breakpoint
```

Network Tab Debugging

```
// Monitor API calls
fetch('https://api.example.com/users')
  .then(response => {
    console.log('Status:', response.status);
    console.log('Headers:', [...response.headers]);
    return response.json();
  })
  .then(data => console.log('Data:', data))
  .catch(error => console.error('Error:', error));

// In Network tab:
// - Filter by XHR/Fetch
// - Click request to see headers, payload, response
// - Check timing for performance issues
```

DOM Inspection

```
<div id="app">
  <button id="toggle">Toggle</button>
  <div id="content" class="hidden">Content</div>
</div>

<script>
  document.getElementById('toggle').addEventListener('click', function() {
    const content = document.getElementById('content');

    // In Elements panel:
    // - See live DOM updates
    // - Edit HTML/CSS directly
    // - Force element state (:hover, :focus)
    // - View computed styles

    content.classList.toggle('hidden');
    console.log('Classes:', content.className);
  });
</script>
```

Performance Profiling

```
// Measure function performance
function heavyComputation() {
  const start = performance.now();

  let sum = 0;
  for (let i = 0; i < 1000000; i++) {
    sum += Math.sqrt(i);
  }

  const end = performance.now();
  console.log(`Execution time: ${end - start}ms`);
  return sum;
}

// In Performance tab:
// 1. Click Record
// 2. Perform action
// 3. Stop recording
// 4. Analyze flame graph
```

Memory Debugging

```
// Detect memory leaks
let leakyArray = [];

function addData() {
  // This creates a memory leak if called repeatedly
  for (let i = 0; i < 1000; i++) {
    leakyArray.push(new Array(1000).fill('data'));
  }
}

// In Memory tab:
// 1. Take heap snapshot before
// 2. Run function
// 3. Take heap snapshot after
// 4. Compare to find leaked objects
```

Part 5: Error Handling in JavaScript

Types of Errors

Error Type	Description	Example
<code>SyntaxError</code>	Invalid code syntax	<code>let x = {</code>
<code>ReferenceError</code>	Variable not defined	<code>console.log(x)</code>
<code>TypeError</code>	Wrong type operation	<code>null.toString()</code>
<code>RangeError</code>	Number out of range	<code>new Array(-1)</code>
<code>URIError</code>	Invalid URI encoding	<code>decodeURI('%')</code>

Try...Catch...Finally

```
try {  
  // Code that might throw an error  
  const result = riskyOperation();  
  console.log(result);  
} catch (error) {  
  // Handle the error  
  console.error('An error occurred:', error.message);  
  console.error('Stack trace:', error.stack);  
} finally {  
  // Always executes (cleanup)  
  console.log('Cleanup complete');  
}
```

Try...Catch Example

```
function divide(a, b) {  
  try {  
    if (b === 0) {  
      throw new Error('Division by zero is not allowed');  
    }  
    if (typeof a !== 'number' || typeof b !== 'number') {  
      throw new TypeError('Both arguments must be numbers');  
    }  
    return a / b;  
  } catch (error) {  
    console.error(`Error: ${error.name}: ${error.message}`);  
    return null;  
  } finally {  
    console.log(`Division attempted: ${a} / ${b}`);  
  }  
}
```

```
console.log(divide(10, 2)); // 5  
console.log(divide(10, 0)); // Error, returns null  
console.log(divide(10, 'a')); // TypeError, returns null
```

Throwing Custom Errors

```
class ValidationError extends Error {
  constructor(field, message) {
    super(message);
    this.name = 'ValidationError';
    this.field = field;
  }
}

class NetworkError extends Error {
  constructor(status, message) {
    super(message);
    this.name = 'NetworkError';
    this.status = status;
  }
}

function validateUser(user) {
  if (!user.email) {
    throw new ValidationError('email', 'Email is required');
  }
  if (!user.email.includes('@')) {
    throw new ValidationError('email', 'Invalid email format');
  }
  if (!user.age || user.age < 0) {
    throw new ValidationError('age', 'Age must be positive');
  }
}

try {
  validateUser({ email: 'invalid', age: -5 });
} catch (error) {
  if (error instanceof ValidationError) {
    console.error(`${error.field}: ${error.message}`);
  }
}
```

Async Error Handling

```
// With Promises
function fetchUser(userId) {
  return fetch(`/api/users/${userId}`)
    .then(response => {
      if (!response.ok) {
        throw new Error(`HTTP ${response.status}`);
      }
      return response.json();
    });
}

// Using .catch()
fetchUser(123)
  .then(user => console.log(user))
  .catch(error => console.error('Failed:', error.message));

// Using async/await with try/catch
async function getUser(userId) {
  try {
    const user = await fetchUser(userId);
    console.log('User:', user);
    return user;
  } catch (error) {
    console.error('Could not fetch user:', error);
    // Re-throw or return default
    return null;
  }
}
```

Error Handling in Event Listeners

```
document.getElementById('submitBtn').addEventListener('click', async function() {  
  try {  
    const formData = getFormData();  
    validateForm(formData);  
  
    const result = await submitForm(formData);  
    showSuccess('Form submitted!');  
  } catch (error) {  
    if (error instanceof ValidationError) {  
      showFieldError(error.field, error.message);  
    } else if (error instanceof NetworkError) {  
      showError('Network issue. Please try again.');
```

Global Error Handling

```
// Catch unhandled errors
window.addEventListener('error', function(event) {
  console.error('Global error:', event.error);
  console.error('File:', event.filename);
  console.error('Line:', event.lineno);
  console.error('Column:', event.colno);

  // Log to error tracking service
  // sendToErrorTracker(event.error);

  // Prevent default browser error display
  event.preventDefault();
});

// Catch unhandled promise rejections
window.addEventListener('unhandledrejection', function(event) {
  console.error('Unhandled promise rejection:', event.reason);
  event.preventDefault();
});
```

Part 6: Introduction to Unit Testing Concepts

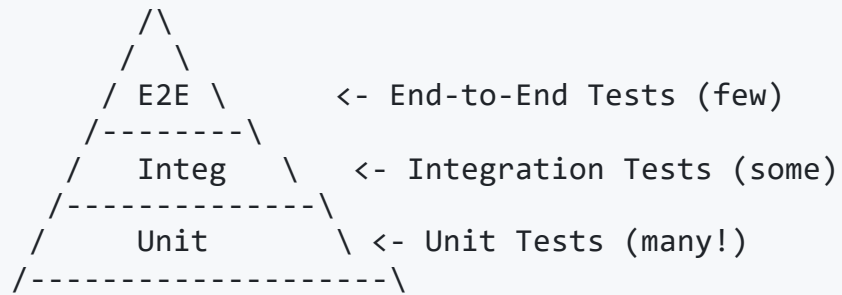
What is Unit Testing?

- **Unit Test:** Tests a single "unit" of code in isolation
- A unit is typically a function or method
- Tests verify that code behaves as expected
- Automated and repeatable

Benefits:

- Catch bugs early
- Enable safe refactoring
- Document expected behavior
- Improve code design

Testing Pyramid



- **Unit Tests:** Fast, isolated, many
- **Integration Tests:** Test components together
- **E2E Tests:** Test full user workflows

AAA Pattern

```
// Arrange - Set up the test
// Act      - Execute the code
// Assert    - Verify the result

function add(a, b) {
  return a + b;
}

// Test case
function testAdd() {
  // Arrange
  const a = 2;
  const b = 3;
  const expected = 5;

  // Act
  const result = add(a, b);

  // Assert
  if (result !== expected) {
    throw new Error(`Expected ${expected}, got ${result}`);
  }
  console.log('✓ add(2, 3) = 5');
}

testAdd();
```

Manual Testing Framework

```
// Simple test runner
function test(description, fn) {
  try {
    fn();
    console.log(`✓ ${description}`);
  } catch (error) {
    console.error(`X ${description}`);
    console.error(`  ${error.message}`);
  }
}

function expect(actual) {
  return {
    toBe(expected) {
      if (actual !== expected) {
        throw new Error(`Expected ${expected}, got ${actual}`);
      }
    },
    toEqual(expected) {
      if (JSON.stringify(actual) !== JSON.stringify(expected)) {
        throw new Error(`Expected ${JSON.stringify(expected)}, got ${JSON.stringify(actual)}`);
      }
    },
    toBeTruthy() {
      if (!actual) {
        throw new Error(`Expected truthy, got ${actual}`);
      }
    },
    toThrow() {
      let threw = false;
      try { actual(); } catch (e) { threw = true; }
      if (!threw) throw new Error('Expected function to throw');
    }
  };
}
```

Writing Tests with Our Framework

```
// Functions to test
function multiply(a, b) {
  return a * b;
}

function divide(a, b) {
  if (b === 0) throw new Error('Division by zero');
  return a / b;
}

function isEven(n) {
  return n % 2 === 0;
}

// Tests
test('multiply 2 * 3 equals 6', () => {
  expect(multiply(2, 3)).toBe(6);
});

test('multiply with zero', () => {
  expect(multiply(5, 0)).toBe(0);
});

test('divide 10 / 2 equals 5', () => {
  expect(divide(10, 2)).toBe(5);
});

test('divide by zero throws error', () => {
  expect(() => divide(10, 0)).toThrow();
});

test('isEven returns true for even numbers', () => {
  expect(isEven(4)).toBeTruthy();
});
```

Jest - Popular Testing Framework

```
// Install: npm install --save-dev jest

// math.js
function add(a, b) {
  return a + b;
}
function subtract(a, b) {
  return a - b;
}
module.exports = { add, subtract };

// math.test.js
const { add, subtract } = require('./math');

describe('Math operations', () => {
  test('adds 1 + 2 to equal 3', () => {
    expect(add(1, 2)).toBe(3);
  });

  test('subtracts 5 - 3 to equal 2', () => {
    expect(subtract(5, 3)).toBe(2);
  });

  test('adds negative numbers', () => {
    expect(add(-1, -2)).toBe(-3);
  });
});
```

Jest Matchers

```
test('various matchers', () => {  
  // Equality  
  expect(value).toBe(exact);           // ===  
  expect(value).toEqual(object);       // deep equality  
  
  // Truthiness  
  expect(value).toBeTruthy();  
  expect(value).toBeFalsy();  
  expect(value).toBeNull();  
  expect(value).toBeUndefined();  
  
  // Numbers  
  expect(value).toBeGreaterThan(5);  
  expect(value).toBeGreaterThanOrEqual(5);  
  expect(value).toBeLessThan(10);  
  expect(value).toBeCloseTo(0.3, 5);  // floating point  
  
  // Strings  
  expect(string).toMatch(/regex/);  
  expect(string).toContain('substring');  
  
  // Arrays  
  expect(array).toContain(item);  
  expect(array).toHaveLength(3);  
  
  // Objects  
  expect(obj).toHaveProperty('name');  
  expect(obj).toHaveProperty('age', 25);  
  
  // Exceptions  
  expect(() => { throw new Error(); }).toThrow();  
});
```

Testing Async Code

```
// fetchData.js
async function fetchUser(id) {
  const response = await fetch(`/api/users/${id}`);
  if (!response.ok) throw new Error('Failed');
  return response.json();
}

// fetchData.test.js
const { fetchUser } = require('./fetchData');

// Using async/await
test('fetches user data', async () => {
  const user = await fetchUser(1);
  expect(user).toHaveProperty('id', 1);
  expect(user).toHaveProperty('name');
});

// Using resolves/rejects
test('fetches user successfully', () => {
  return expect(fetchUser(1)).resolves.toHaveProperty('name');
});

test('handles fetch error', () => {
  return expect(fetchUser(999)).rejects.toThrow('Failed');
});
```

Testing DOM Manipulation

```
// domUtils.js
function toggleVisibility(element) {
  element.classList.toggle('hidden');
  return element.classList.contains('hidden');
}

function updateText(element, text) {
  element.textContent = text;
}

module.exports = { toggleVisibility, updateText };

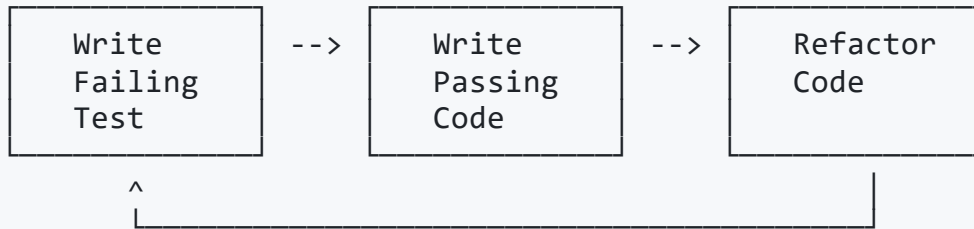
// domUtils.test.js
/**
 * @jest-environment jsdom
 */
const { toggleVisibility, updateText } = require('./domUtils');

describe('DOM Utilities', () => {
  beforeEach(() => {
    document.body.innerHTML = `
      <div id="test" class="visible">Test</div>
    `;
  });

  test('toggleVisibility adds hidden class', () => {
    const el = document.getElementById('test');
    const isHidden = toggleVisibility(el);
    expect(isHidden).toBe(true);
    expect(el.classList.contains('hidden')).toBe(true);
  });

  test('updateText changes content', () => {
    const el = document.getElementById('test');
    updateText(el, 'New Text');
    expect(el.textContent).toBe('New Text');
  });
});
```

Test-Driven Development (TDD)



Red-Green-Refactor Cycle:

1. **Red:** Write a test that fails
2. **Green:** Write minimal code to pass
3. **Refactor:** Clean up the code

TDD Example

```
// Step 1: Write failing test (Red)
// calculator.test.js
test('calculates sum of array', () => {
  expect(sum([1, 2, 3])).toBe(6);
});
// → FAIL: sum is not defined

// Step 2: Write minimal code (Green)
// calculator.js
function sum(numbers) {
  return numbers.reduce((a, b) => a + b, 0);
}
module.exports = { sum };
// → PASS

// Step 3: Add more tests
test('sum of empty array is 0', () => {
  expect(sum([])).toBe(0);
});

test('sum of single number', () => {
  expect(sum([5])).toBe(5);
});
```

Best Practices for Testing

```
// 1. Test one thing at a time
test('validates email format', () => {
  expect(validateEmail('user@example.com')).toBe(true);
});

test('rejects invalid email', () => {
  expect(validateEmail('invalid')).toBe(false);
});

// 2. Use descriptive test names
test('should return user object with id and name properties', () => {
  // ...
});

// 3. Arrange-Act-Assert clearly
test('calculates discount correctly', () => {
  // Arrange
  const price = 100;
  const discount = 0.2;

  // Act
  const result = calculateDiscount(price, discount);

  // Assert
  expect(result).toBe(80);
});

// 4. Test edge cases
test('handles empty input', () => { /* ... */ });
test('handles maximum value', () => { /* ... */ });
test('handles negative numbers', () => { /* ... */ });
```

Code Coverage

```
# Run tests with coverage
npx jest --coverage

# Coverage report shows:
# - Statements: % of statements executed
# - Branches:   % of conditional branches
# - Functions:  % of functions called
# - Lines:      % of lines executed
```

Goal: Aim for 70-80%+ coverage, but focus on meaningful tests over numbers.

Summary

Topic	Key Takeaways
DOM Events	<code>addEventListener()</code> , event object, delegation
Form Events	<code>submit</code> , <code>change</code> , <code>input</code> , <code>focus</code> , <code>blur</code>
Keyboard/Mouse	<code>keydown</code> , <code>click</code> , <code>mousemove</code> , drag & drop
Form Validation	HTML5 + JS validation, Constraint API
Git	init, add, commit, branch, merge, push, pull
Debugging	console, breakpoints, watch, network, performance
Error Handling	try/catch, custom errors, async handling
Unit Testing	AAA pattern, Jest, TDD, coverage

Thank You!

Questions?

Resources:

- MDN Web Docs: developer.mozilla.org
- Git Documentation: git-scm.com/doc
- Jest: jestjs.io
- Chrome DevTools: developer.chrome.com/docs/devtools